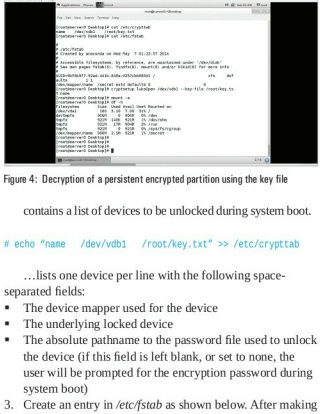

Figure 4: Decryption of a persistent encrypted partition using the key file

contains a list of devices to be unlocked during system boot.

```
# echo "name   /dev/vdb1   /root/key.txt" >> /etc/cryptab
```

…lists one device per line with the following space-separated fields:

- The device mapper used for the device
- The underlying locked device
- The absolute pathname to the password file used to unlock the device (if this field is left blank, or set to none, the user will be prompted for the encryption password during system boot)

3. Create an entry in */etc/fstab* as shown below. After making the entry in */etc/fstab*, if we open the partition using the key file, the command will be:

```
# cryptsetup luksOpen /dev/vdb1  --key-file /root/key.txt name
```

As shown in the entry of the *fstab* file, if the device to be mounted is named, then the file system on which the encrypted partition should be permanently mounted is in the other entries. Also, no passphrase is asked for separately now, as we have supplied the key file, which has already been added to the partition. The partition can now be mounted using the *mount -a* command, after which the mounted partition can be verified upon reboot by using the *df -h* command. All the steps are clearly described in Figure 5.

Figure 5: Available slots for an encrypted partition are shown

Note: The device listed in the first field of */etc/fstab* must match the name chosen for the local name to map in */etc/crypttab*. This is a common configuration error.

Attaching a key file to the desired slot

LUKS offers a total of eight key slots for encrypted devices (0-7). If other keys or a passphrase exist, they can be used to open the partition. We can check all the available slots by using the luksDump command as shown below:

```
# cryptsetup luksDump /dev/vdb1
```

As can be seen in Figure 5, Slot0 and Slot1 are enabled. So the key file we have supplied manually, by default moves to Slot1, which we can use for decrypting the partition. Slot0 carries the master key, which is supplied while creating the encrypted partition.

Now we will add a key file to Slot3. For this, we have to generate a key file of random numbers by using the *urandom* command, after which we will add it to Slot3 as shown below. The passphrase of the encrypted partition must be supplied in order to add any secondary key to the encrypted volume.

```
# dd if=/dev/urandom of=/root/key2.txt bs=4096 count=1.
#cryptsetup luksAddKey /dev/vdb1 --key-slot 3  /root/key2.
txt.
```

After adding the secondary key, again run the *luksDump* command to verify whether the key file has been added to Slot3 or not. As shown in Figure 7, the key file has been added to Slot3, as Slot2 remains disabled and Slot3 has been

Figure 6: Secondary key file *key2.txt* has been added at Slot3

Figure 7: Slot3 enabled successfully